| | | | |
|---|---|---|---|
| **Name of Student:**<br>VIKAS FOOLCHAND KUSHWAHA | | | |
| **Roll Number:**<br>29/B | | **Lab Practical Number:**<br>02 | |
| **Title of LAB Practical:**<br>To implement Python programs with conditional and loops. | | | |
| **DOP:**<br>29/10/2026 | | **DOS:**<br>12/11/2026 | |
| **CO Mapped:**<br>CO1 | **PO Mapped:**<br>PO3, PO4,<br>PSO1, PSO2 | **Faculty Signature:** | **Marks:** |

**Aim**: To implement Python programs with conditional and loops.

# Description:

## Python Conditionals

### If-statement

In Python, we use conditional statements to test for various conditions. They are defined using the **if** keyword and are known as **if-statements**. The syntax for it is defined as follows:

```
if condition:
    statement_1
    statement_2
    …
```

Note the indentation after the first line of code block as it is necessary for python to evaluate them accurately.

A **condition** can be any python expression that evaluates to either **true** and **false**. Based on the result, the statements are evaluated. If the condition evaluates to true, the statements within the if block are executed. If it evaluates to false, the statements are skipped and the interpreter jumps to the line of code that follows the if statement. Often times, conditionals are comparison statements defined using the comparison operators. They are derived from mathematic and their usage is given below.

| **Operator Name** | **Operator Symbol** |
|---|---|
| Equals | a == b |
| Not Equals | a != b |
| Less than | a < b |

| | |
|---|---|
| Greater than | a > b |
| Less than and equal to | a <= b |
| Greater than and equal to | a >= b |

## If-Else Block

An if-statement can follow with an **else** block. If the condition evaluates to false, the statements in the else block are executed. Such statements are called if-else statements.

```
if condition:
    true_statement
else:
    false_statement
```

## Nested if-else

The if-else block can be converted into a nested if-else by adding multiple if statements within the same if block.

```
if condition_1:
    # evaluate only if condition_1 is true
    statement_1

    if condition_2:
        # evaluate only if condition_2 is true
        statement_2

    # continue executing other statements of condition_1
    statement_3
```

## If-Else Ladder

Multiple if statements can be chained together with the python **elif** keyword. This helps to evaluate multiple expressions at once and only execute those

```
if condition_1:
    statement_1

elif condition_2:
    statement_2

elif condition_3:
    statement_3

else:
    statement_4
```

# Python Loops

The two widely used loop statements in python are **while** and **for** loop. They iterate over a set of values given by a list or collection and repeatedly execute a set of statements as given by the user. They are explained below.

## For loop

The following is the syntax for **'for'** loop.

```
for var in list:
    statement_a1
    statement_a2
    …
```

The loop iterates over each and every value of the **list** and stores them into a variable as defined at the location of **var**. It can be of any name. The loop will keep executing every statement inside the for block, line by line until each and every element of the **list** has been iterated over or consumed.

## Else-block and control flow

Python also supports the **else** keyword which can be followed by or incorporated into the for statement. The purpose of this keyword is to test for breaks that are often inserted using the **break** keyword. If the loop doesn't break and continues to execute till the end, then the

statements in the else block are executed. The break statement is often guarded with a conditional i.e. using the if-statement. The syntax is as follows:

```
for var in list:
    statement_a1

    if condition:
        break

    statement_a2
    …

else:
    statement_b1
    statement_b2
    …
```

If the condition in the nested if-statement evaluates to true, the loop will end right there without evaluating further elements of the list or the else block. If not, then the loop continues till the end and the statements in the else block are also executed.

Some other keywords like **continue** and **pass** are also supported. They are similar to the **break** keyword in that they manage the control flow of the executing loop. These are called control-flow statements.

## While loop

Similar to for loop, another loop is supported in python called the **while** loop. Instead of iterating over elements on its own, it evaluates a condition before every execution loop. If the condition evaluates to true, it continues executing the statements. If not, it exits from the control flow and passes on the executing flow to the next line of code proceeding the while block.

The syntax for while loop is given below.

```
while condition:
    statement_a1

    if condition:
        break
```

```
    statement_a2
    …

else:
    statement_b1
    statement_b2
    …
```

Similar to the for loop, the while loop also supports the **else** block.

# Programs:

**QUESTION 1**: **To find all the prime numbers in the interval 0 to 100.**

**CODE**

```
print("Prime numbers from 0 to 100")

for number in range(1, 101):
    is_prime = True
    for divisor in range(2, int(number ** 0.5) + 1):
        if number % divisor == 0:
            is_prime = False
            break
    if is_prime:
        print(number, end="  ")
```

**OUTPUT**

```
> python Q1_prime_numbers.py
Prime numbers from 0 to 100
1  2  3  5  7  11  13  17  19  23  29  31  37  41  43  47  53  59  61  67
  71  73  79  83  89  97
```

**QUESTION 2**: **To check if the given number is Armstrong no or not.**

**CODE**

```
number = int(input("Enter a number: "))

digits = str(number)
power = len(digits)
total = 0

for digit in digits:
    total += int(digit) ** power

if total == number:
    print(f"{number} is an armstrong number.")
else:
    print(f"{number} is not an armstrong number.")
```

**OUTPUT**

```
[14:33] vikas@shadow [0] ~dv/labs/PPLP/PPLP2
> python Q2_armstrong_test.py
Enter a number: 226
226 is not an armstrong number.
```

```
> python Q2_armstrong_test.py
Enter a number: 153
153 is an armstrong number.
```

**QUESTION 3**: **To check if the given char is vowel or consonant.**

**CODE**

```
character = input("Enter a character: ")

if len(character) != 1 or not character.isalpha():
    print("Please enter an alphabetic character.")
elif character in "aeiou":
    print(character, "is a vowel.")
else:
    print(character, "is a consonant.")
```

**OUTPUT**

```
> python Q3_vowels.py
Enter a character: x
x is a consonant.
```

```
> python Q3_vowels.py
Enter a character: a
a is a vowel.
```

**QUESTION 4**: **Write a Program to Take in the Marks of 3 Subjects and Display the Grade.**

**CODE**

```
import sys
```

```
mark1 = float(input("Enter marks for Subject 1: "))
mark2 = float(input("Enter marks for Subject 2: "))
mark3 = float(input("Enter marks for Subject 3: "))

average = (mark1 + mark2 + mark3) / 3
print(f"Your average marks is {average: .3f}.")

if average > 90:
    grade = "O"
elif average > 85:
    grade = "A+"
elif average > 80:
    grade = "A"
elif average > 75:
    grade = "B+"
elif average > 70:
    grade = "B"
elif average > 65:
    grade = "C+"
elif average > 60:
    grade = "C"
elif average > 55:
    grade = "D+"
elif average > 50:
    grade = "D"
elif average > 45:
    grade = "E+"
elif average > 40:
    grade = "E"
else:
    print("Oops! You failed the exam.")
    sys.exit()

print(f"You have scored {grade} grade.")
```

**OUTPUT**

```
> python Q4_grade.py
Enter marks for Subject 1: 25
Enter marks for Subject 2: 45
Enter marks for Subject 3: 40
Your average marks is  36.667.
Oops! You failed the exam.
```

```
> python Q4_grade.py
Enter marks for Subject 1: 55
Enter marks for Subject 2: 60
Enter marks for Subject 3: 72
Your average marks is  62.333.
You have scored C grade.
```

**QUESTION 5**: **To add two matrices.**

**CODE**

```
print("# For Matrix A")
rows = int(input("Enter the number of rows: "))

# TODO: Convert all elements of matrix to integer

matrix1 = []
for i in range(rows):
    matrix1.append([int(n) for n in input(f"Enter row {i}: ").split("
")])

print("\n# For Matrix B")

matrix2 = []
for i in range(rows):
    matrix2.append([int(n) for n in input(f"Enter row {i}: ").split("
")])
```

```
matrix_total = []
cols = len(matrix1[0])

for i in range(rows):
    row_total = []
    for j in range(cols):
        row_total.append(matrix1[i][j] + matrix2[i][j])
    matrix_total.append(row_total)

print("Sum of the two matrices:")
for row_total in matrix_total:
    print(row_total)
```

**OUTPUT**

```
> python Q5_matrix_addition.py
# For Matrix A
Enter the number of rows: 2
Enter row 0: 1 2
Enter row 1: 3 4

# For Matrix B
Enter row 0: 5 6
Enter row 1: 7 8
Sum of the two matrices:
[6, 8]
[10, 12]
```

**<u>QUESTION 6</u>: To convert the month name to a number of days.**

**CODE**

```
import calendar
```

```
import sys

month_name = input("Enter a month name: ").strip().lower()

months = {number: name for number, name in \
    enumerate(calendar.month_name) if name}

for month in months.items():
    if month_name in month[1].lower():
        break
else:
    print("Invalid month name")
    sys.exit()

if month[0] == 2:
    print("Note: February has 28, or 29 in the leap year.")
else:
     print(f"{month[1]} has {calendar.monthrange(2026, month[0])[1]}
days.")
```

**OUTPUT**

```
> python Q6_calendar.py
Enter a month name: jan
January has 31 days.
```

```
> python Q6_calendar.py
Enter a month name: feb
Note: February has 28, or 29 in the leap year.
```

**<u>QUESTION 7</u>: To check the validity of password input by users.**

**Validation:**

- At least 1 letter between [a-z] and 1 letter between [A-Z].

- At least 1 number between [0-9].
- At least 1 character from [$#@].
- Minimum length 6 characters.
- Maximum length 16 characters.

**CODE**

```
import re

password = input("Enter a password: ")

valid = (
    6 <= len(password) <= 16
    and re.search(r"[a-z]", password)
    and re.search(r"[A-Z]", password)
    and re.search(r"[0-9]", password)
    and re.search(r"[$#@]", password)
)

if valid:
    print("Valid Password.")
else:
    print("Invalid Password")
    print("Password should be of 6 to 16 characters long.")
    print("It should include:")
    print("- atleast one lowercase letter")
    print("- atleast one uppercase letter")
    print("- atleast one number letter")
    print("- atleast one symbol from $, # or @")
```

**OUTPUT**

```
> python Q7_password_validation.py
Enter a password: hello
Invalid Password
Password should be of 6 to 16 characters long.
It should include:
- atleast one lowercase letter
- atleast one uppercase letter
- atleast one number letter
- atleast one symbol from $, # or @
```

```
> python Q7_password_validation.py
Enter a password: P@ssw0rd
Valid Password.
```

**QUESTION 8**: **To check if a number is palindrome or not.**

**CODE**

```
number = input("Enter a number: ")

if number.isdigit() and number == number[::-1]:
    print(f"{number} is a Palindrome.")
else:
    print(f"{number} is not a Palindrome.")
```

**OUTPUT**

```
> python Q8_palindrome.py
Enter a number: 123
123 is not a Palindrome.
```

```
> python Q8_palindrome.py
Enter a number: 121
121 is Palindrome.
```

# Conclusion:

In this Practical, we learned about Python Conditionals and Loops and various control flow mechanisms that are used to solve and implement solutions to various kinds of mathematical problems and real world concerns. The programs here perfectly demonstrate the power and the usefulness of control flow statements in day to day programming.